

Question (a)

Value Iteration is a fundamental algorithm in Reinforcement Learning (RL) used to find the optimal policy for a Markov Decision Process. It is based on iteratively updating the value function using the Bellman optimality equation. The algorithm repeatedly evaluates and improves the value of each state until the values converge. Value Iteration Algorithm (Pseudocode) is as follows:

Input: MDP (S, A, P, R, γ), θ (convergence threshold)

Output: Optimal value function V^* and optimal policy π^*

1. **Initialize** $V(s) = 0$ for all $s \in S$

2. **Repeat:**

$\Delta = 0$

For each state $s \in S$:

$$\left| \begin{array}{l} v = V(s) \\ V(s) = \max_a \sum_s' P(s'|s, a) * [R(s, a) + \gamma * V(s')] \\ \Delta = \max(\Delta, |v - V(s)|) \end{array} \right.$$

If $\Delta < \theta$: Break

3. **Derive optimal policy:**

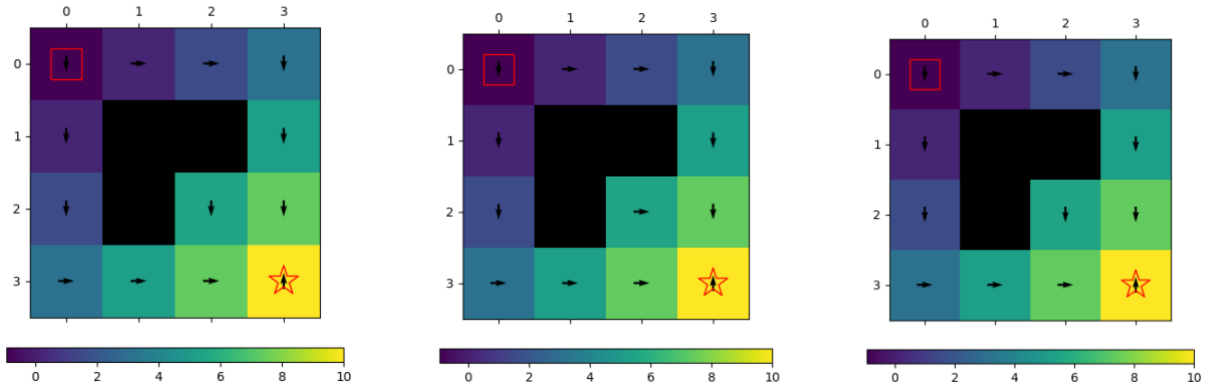
$\pi^*(s) = \operatorname{argmax}_a \sum_s' P(s'|s, a) * [R(s, a) + \gamma * V(s')]$

4. **Return** V^* and π^*

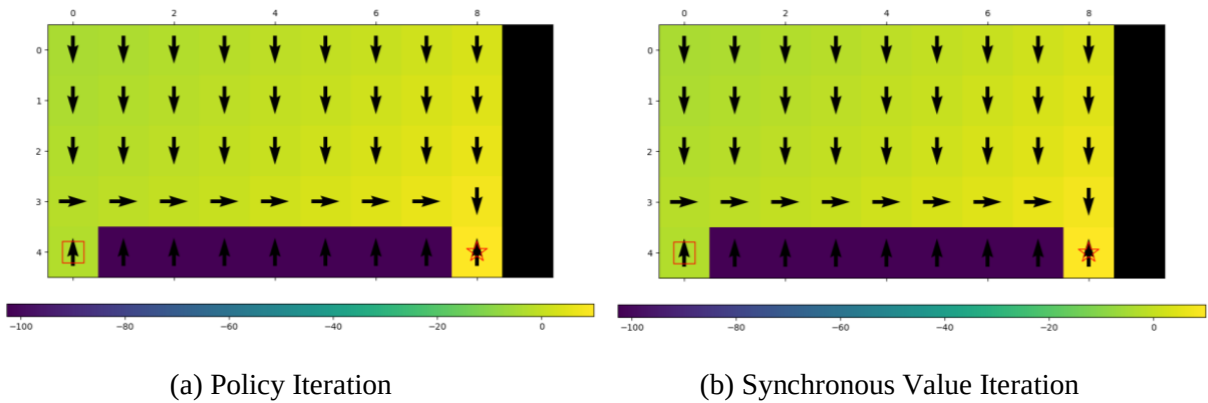
Value Iteration and Policy Iteration are both dynamic programming algorithms used in reinforcement learning to find the optimal policy. Value Iteration directly updates the value function using the Bellman optimality equation and extracts the policy after the values converge. In contrast, Policy Iteration alternates between evaluating a given policy (policy evaluation) and improving it (policy improvement) until convergence.

In Value Iteration (VI), synchronous updates all state values at once, ensuring consistency but requiring more iterations. Asynchronous updates state values individually, potentially speeding up convergence by focusing on key states first but may introduce temporary inconsistencies.

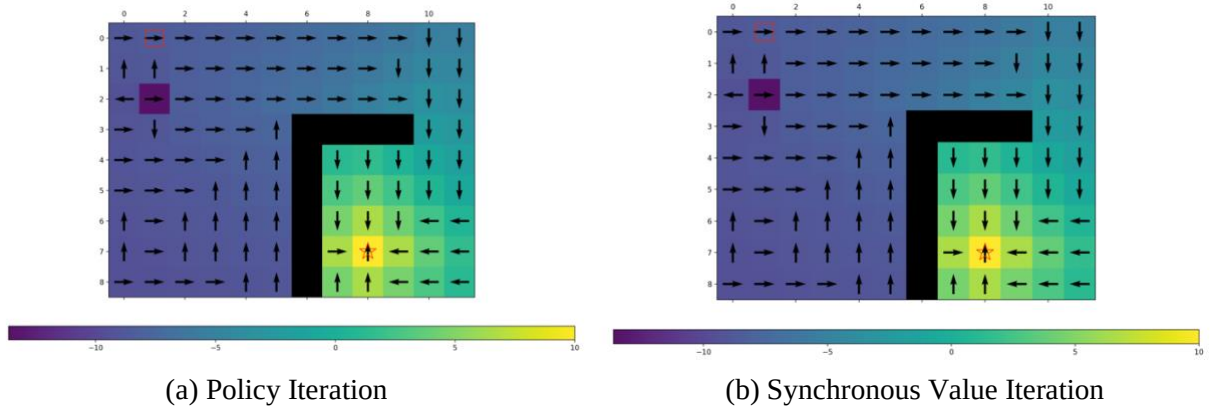
The policy results of Value Iteration and Policy Iteration under different world configurations are as follows.



(a) Policy Iteration (b) Synchronous Value Iteration (c) Asynchronous Value Iteration
Figure 1.1 The policy results generated by different algorithms on small world



(a) Policy Iteration (b) Synchronous Value Iteration
Figure 1.2 The policy results generated by different algorithms on cliff world



(a) Policy Iteration (b) Synchronous Value Iteration
Figure 1.3 The policy results generated by different algorithms on grid world

The images visually represent the state values and the policy in a grid world. The plot shows the value function, where each cell's color corresponds to its value. Darker colors represent lower values, while lighter colors represent higher values. This helps visualize how desirable different states are based on the agent's goal and environment. In summary, the image provides a comprehensive view of both the values associated with each state and the optimal actions to be taken.

It can be observed that in the three different world configurations, both PI (Policy Iteration) and VI (Value Iteration) are able to find the optimal path, yielding similar or identical results.

In Value Iteration, the algorithm stops if the value function changes by less than a predefined threshold, indicating convergence. In this experiment, we set the threshold as 1×10^{-6} . While in policy iteration, the policy improvement stops when the policy no longer changes.

In order to compare the synchronous and asynchronous VI algorithms, a line graph of the maximum difference in the value of each stage after each stage is drawn as follows.

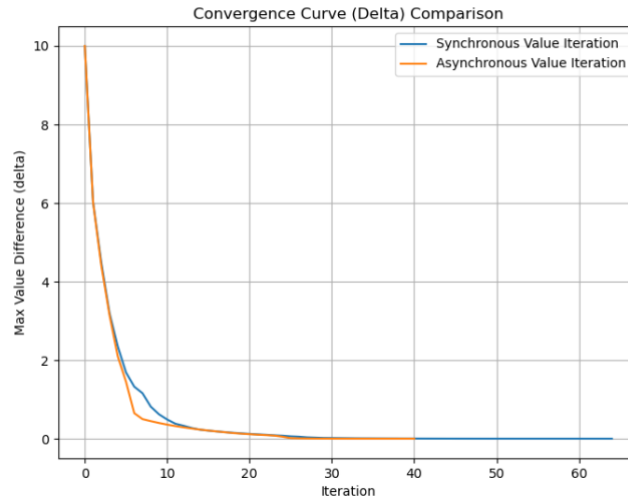
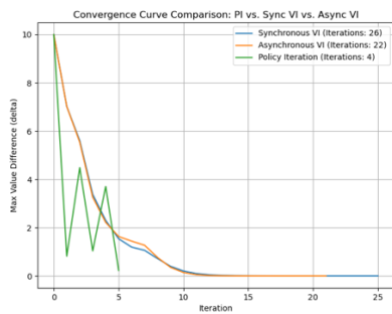


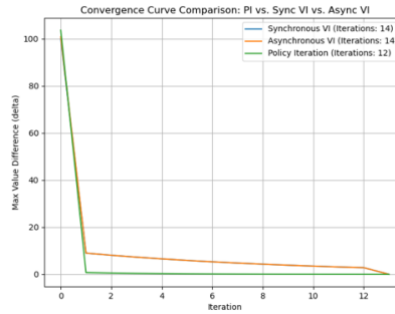
Figure 1.4 Comparison between sync VI and async VI

From the chart, it is clear that synchronous Value Iteration updates all states in each iteration, resulting in a smoother convergence process. On the other hand, asynchronous Value Iteration updates only one state at a time, leading to a more unstable convergence curve. However, it typically reaches the optimal solution more quickly compared to the synchronous approach.

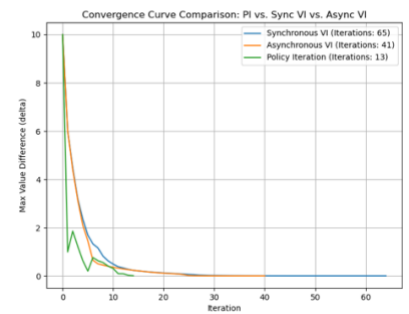
Finally, we evaluate the PI, VI, and VIA algorithms using the same metrics. The x axis recorded the number of iterations needed to converge.



(a) small world



(b) cliff world



(c) grid world

Figure 1.5 Comparison between PI and VI on different world configurations

Each policy evaluation in PI leads to significant policy improvement, quickly approaching the optimal policy. The improvements from each step typically enhance performance, allowing PI to converge in fewer iterations. Unlike PI, each update in VI doesn't always result in major policy improvements, as it calculates the optimal policy by maximizing the value function of the current state. Asynchronous Value Iteration (VIA) generally converges faster than VI but slower than PI. The key idea behind VIA is that it doesn't update all states' value functions simultaneously.

Question (b)

SARSA (State-Action-Reward-State-Action) is an on-policy reinforcement learning algorithm that updates the action-value function (Q-value) based on the action taken by the agent.

```

Initialize  $Q(s, a)$  arbitrarily, for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ 
Repeat (for each episode):
     $E(s, a) = 0$ , for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ 
    Initialize  $S, A$ 
    Repeat (for each step of episode):
        Take action  $A$ , observe  $R, S'$ 
        Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
         $\delta \leftarrow R + \gamma Q(S', A') - Q(S, A)$ 
         $E(S, A) \leftarrow E(S, A) + 1$ 
        For all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ :
             $Q(s, a) \leftarrow Q(s, a) + \alpha \delta E(s, a)$ 
             $E(s, a) \leftarrow \gamma \lambda E(s, a)$ 
         $S \leftarrow S'; A \leftarrow A'$ 
    until  $S$  is terminal

```

Figure 2.1 Process of SARSA algorithm

In order to study the settings of different parameters, the value ranges of different parameters were set in advance and numerical experiments were conducted on them. The experiment was conducted on learning rate alpha, and the results are as follows.

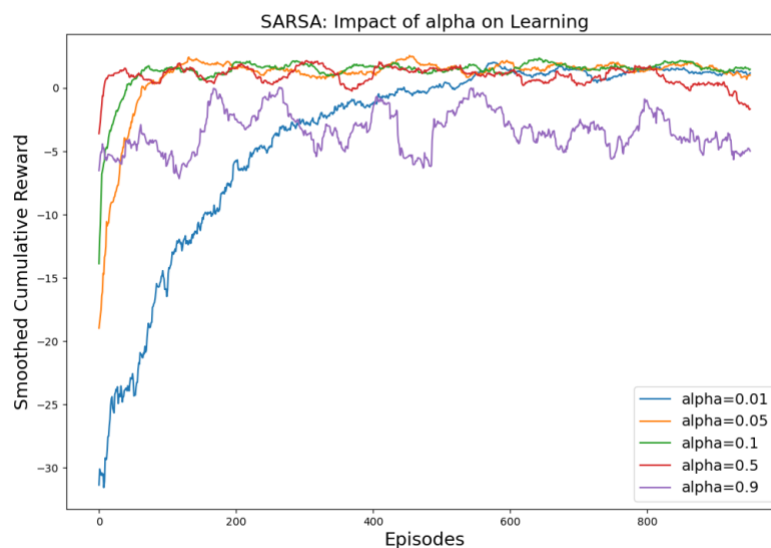


Figure 2.2 Experiment on alpha

The horizontal axis is the number of episodes the algorithm runs, and the vertical axis is the cumulative reward after sliding window smoothing. The larger the value, the better the algorithm's current optimization effect. It can be seen that when alpha is very small (such as 0.01), the algorithm converges slowly and reaches stability in about 600 rounds. When alpha is too large, such as 0.9, the step size of the parameter update is large, resulting in an unstable state, so that there is no way to find a reasonable optimization strategy. Therefore, according to the experimental results, the best alpha parameter is selected as 0.5. The same conclusion can be drawn according to the policy results of different alpha settings.

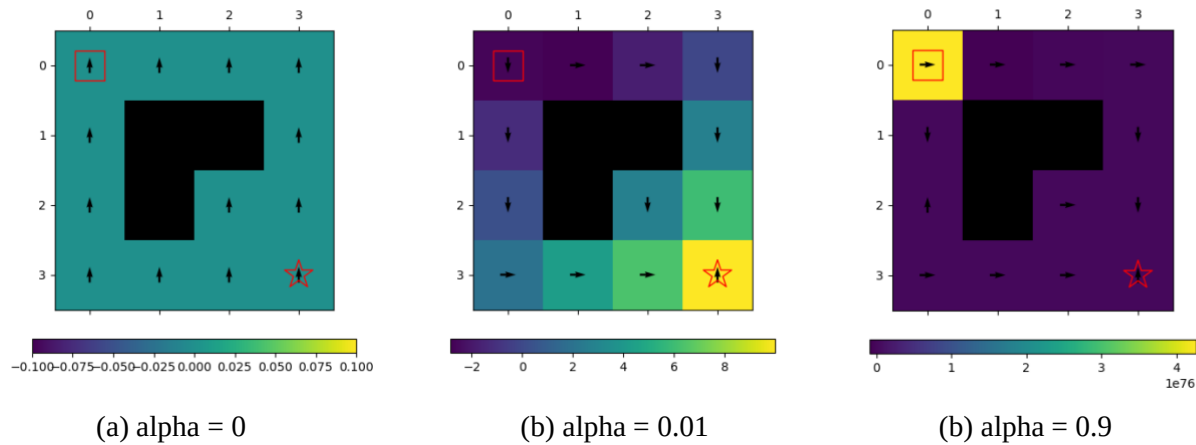


Figure 2.3 Policy result on different alpha

After that, different epsilon values are set to represent the probability that the algorithm will choose a new state for exploration under a certain state.

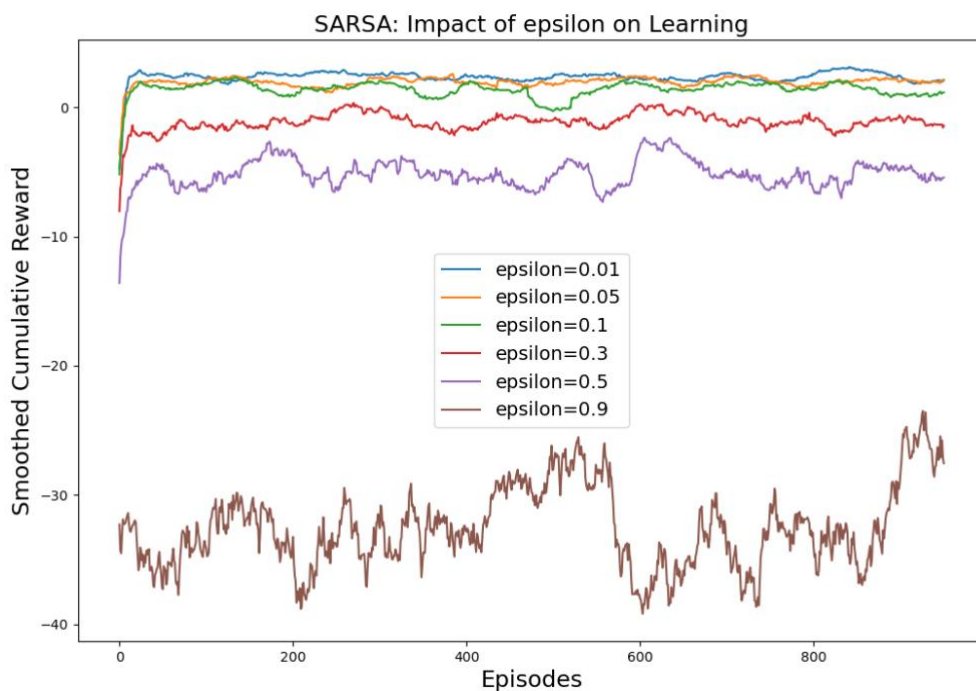


Figure 2.4 Experiment on epsilon

In the SARSA algorithm, the epsilon parameter controls the exploration-exploitation tradeoff. Specifically. When epsilon is large, the agent will tend to choose random actions most of the time, this may lead to slower convergence and may let the algorithm to lose the ability to find optimum result. With a smaller epsilon, the agent almost always exploits the best-known action (based on the current Q-values). It will rarely explore other actions. The algorithm can converge faster but has risk of local optima.

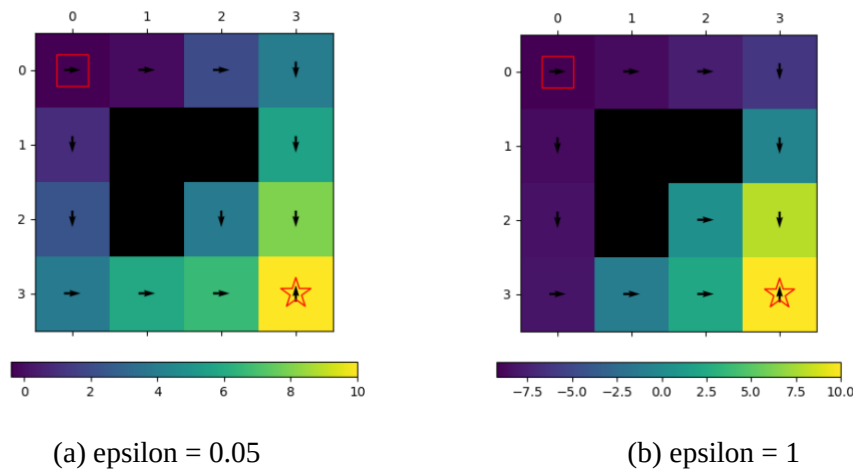


Figure 2.5 Policy result on different epsilon

From the policy result graphs with different epsilon values, we can see that when epsilon is equal to 1, the algorithm will explore a new environment at each step instead of selecting the optimal state, which will cause the algorithm to lose its optimization effect. Based on the results, setting 0.1 for epsilon is better.

For each episode, the algorithm will perform several iterations, and the maximum number of iterations also has a certain impact on the results. The results are as follows when different maximum numbers of iterations are set.

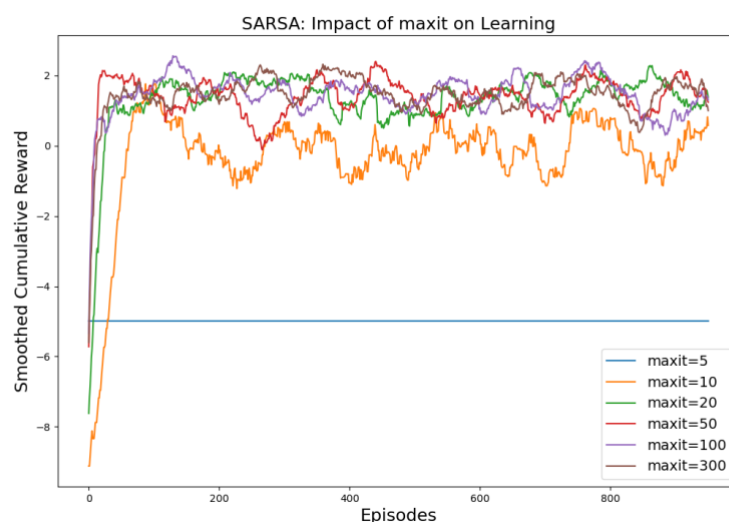
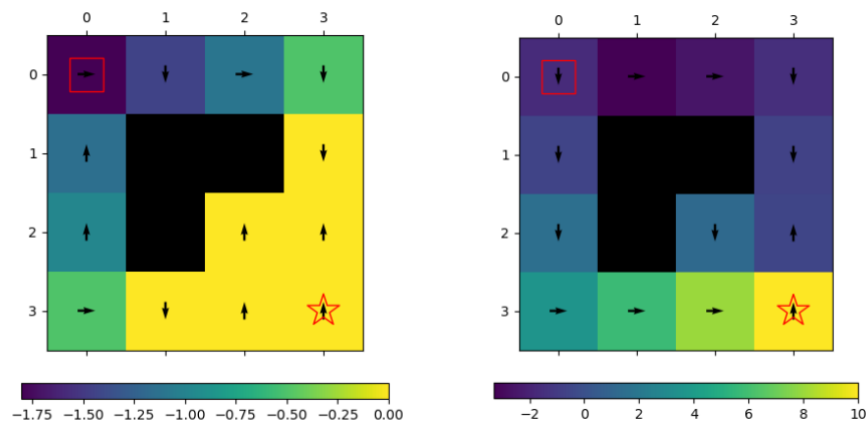


Figure 2.6 Experiment on different max_iteration

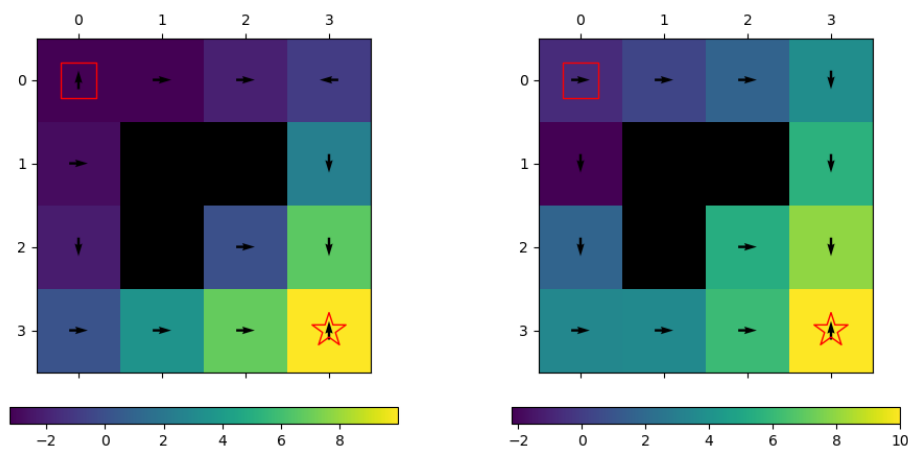
When the maximum number of iterations is very small, such as $\text{maxit}=5$, the algorithm cannot fully update the parameters, resulting in failure to find the optimal solution. When the number of iterations is too large, although the algorithm can fully function, it will consume a lot of resources and computing power. For example, the results of 300 and 50 iterations in the figure are not much different.

The policy is optimized based on the number of episodes and iterations. The results are as follows.



(a) iteration=10, episode=10

(b) iteration=10, episode=100



(c) iteration=100, episode=10.

(d) iteration=100, episode=200

Figure 2.7 Policy results on different max_iteration , episode number

As can be seen from the figure, we need to find the best match between the two parameters. When both are small, the algorithm cannot get the optimal solution because the number of calculations is not enough. Therefore, we finally choose max_iteration as 100 and the number of episodes as 200 to ensure sufficient calculation without consuming too many resources.

Finally, we compare the SARSA and Expected SARSA algorithms. The key difference is that SARSA updates the Q-values based on the current action taken, while Expected SARSA computes the expected Q-value over all possible actions, weighted by their probabilities.

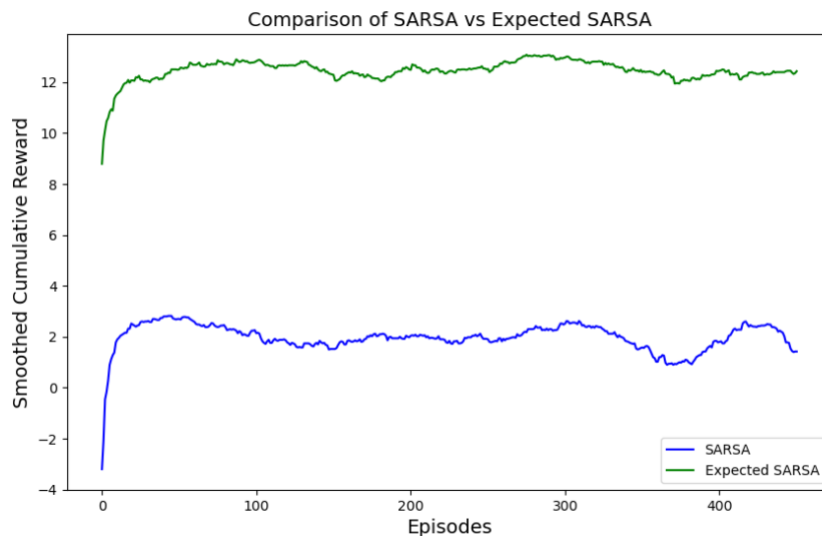


Figure 2.8 Comparison of SARSA and Expected SARSA

In terms of performance, SARSA may experience instability in updates due to over-reliance on the current action, which can slow down convergence. On the other hand, Expected SARSA reduces this fluctuation by considering expected values, leading to a more stable learning process and potentially faster convergence in certain cases. Expected SARSA often leads to better results (higher rewards) than standard SARSA because it accounts for the expected value of the next action rather than just the action actually taken. This provides a more accurate estimate of the value function, as it considers all possible actions and their probabilities instead of relying solely on the next action chosen.

Question (c)

Under the same small world configuration, the Q-Learning algorithm is implemented. The same parameter optimization method as the sarsa algorithm is selected, and the following results are obtained.

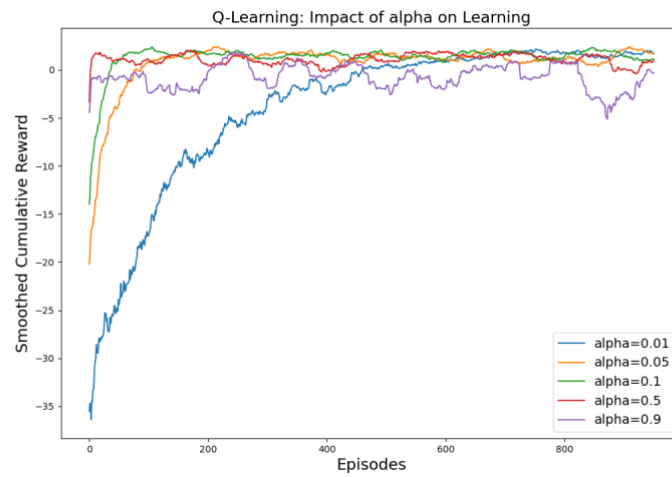


Figure 3.1 Different α settings on Q-Learning

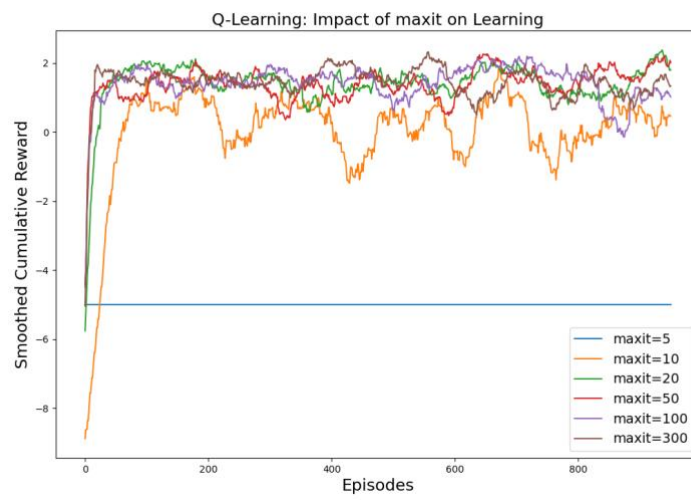


Figure 3.2 Different max_iteration settings on Q-Learning

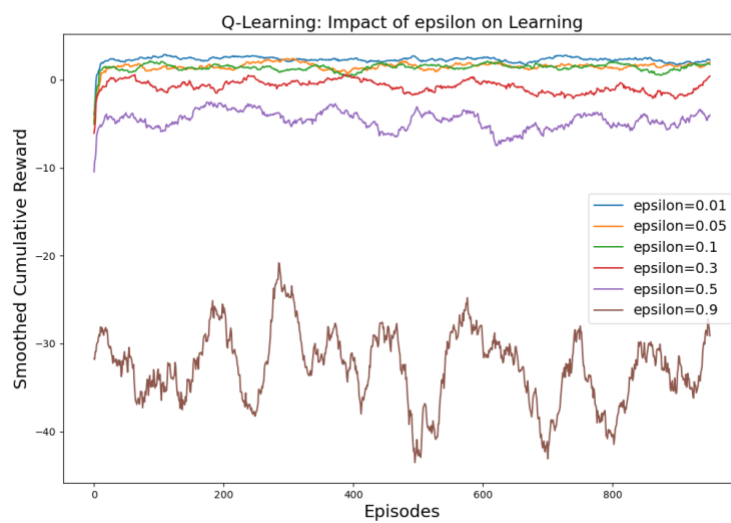


Figure 3.3 Different ϵ settings on Q-Learning

Based on these results, we choose $\alpha = 0.5$, $\epsilon = 0.05$, $\text{max_iteration} = 100$, $\text{episode} = 200$. Finally, we compare the efficiency of the two algorithms and the learned policies.

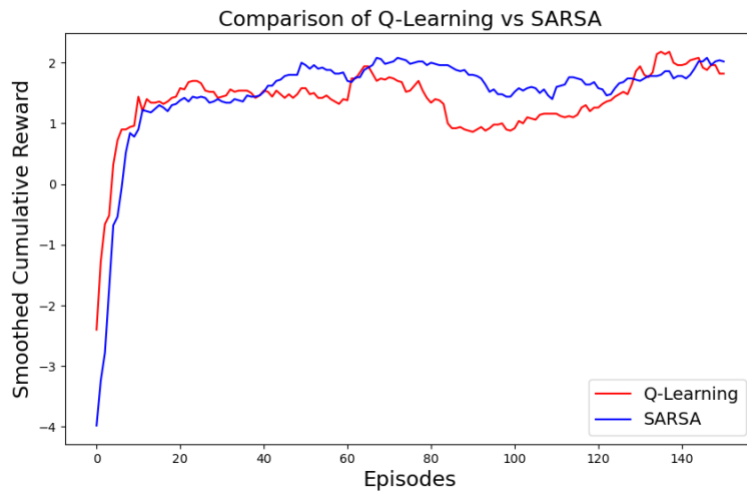


Figure 3.4 Efficiency comparison between Q-Learning and SARSA

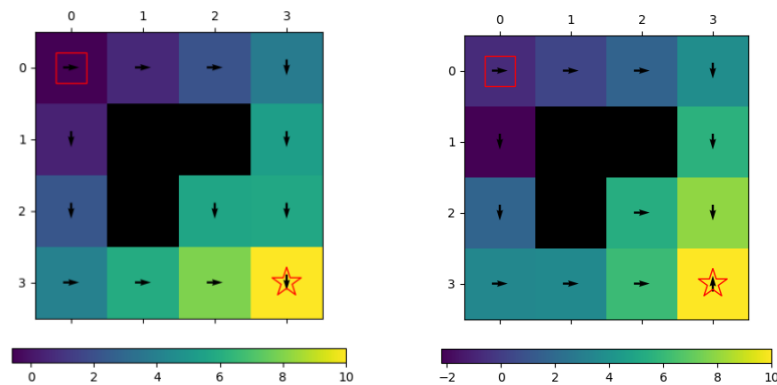


Figure 3.5 Learned policy: Q-Learning (left) and SARSA (right)

It can be seen that in the small world environment, the effects of the two are similar, and Q-learning is more efficient in finding the optimal solution (as reflected in a steeper and earlier rise). Due to faster update speed, Q-Learning typically requires fewer iterations in static environments. SARSA may take longer but is stronger in non-deterministic environments.

SARSA updates based on the actual action taken, making its learning process more cautious but robust, which is suitable for high-risk or dynamic environments. On the other hand, Q-Learning always estimates the optimal action, leading to more aggressive updates but being more sensitive to environmental noise, making it better suited for stable and static environments.

Question (d)

When we extend the SARSA and Q-Learning algorithms to the Cliff World configuration.

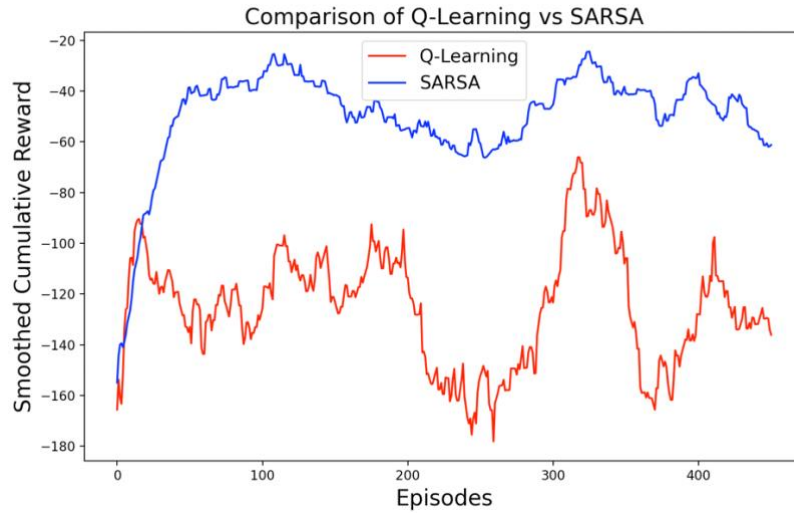


Figure 4.1 Efficiency comparison between Q-Learning and SARSA, $\alpha = 0.5$, $\epsilon = 0.3$

Experimental results reveal that SARSA outperforms Q-Learning in terms of achieving a higher stable cumulative reward. The key reason for this is that SARSA is more conservative in its updates, relying on the actual action taken, which leads to a more cautious learning approach. This cautious nature makes SARSA more robust, particularly in non-deterministic environments like Cliff World, where the state transitions can vary. In contrast, Q-Learning tends to estimate the optimal action aggressively, which can make it more sensitive to environmental noise and, in this case, lead to more unstable learning behavior.

The policies derived by the two algorithms are as follows.

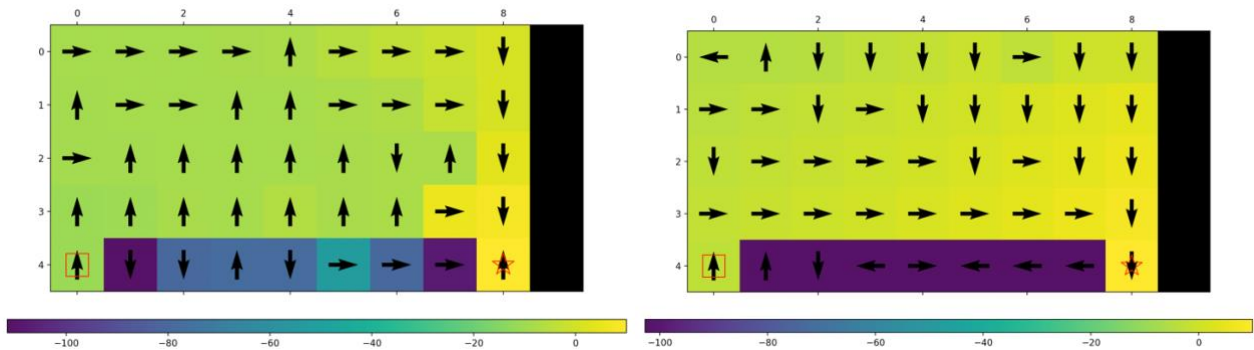


Figure 4.2 Policies by SARSA (left) and Q-Learning (right)

The policy obtained shows that SARSA tends to choose paths farther from the cliff edge, while Q-Learning prefers paths closer to the cliff. As a result, SARSA prioritizes stability and risk-avoidance, while Q-Learning is more driven by immediate reward maximization, even at the cost of stability.

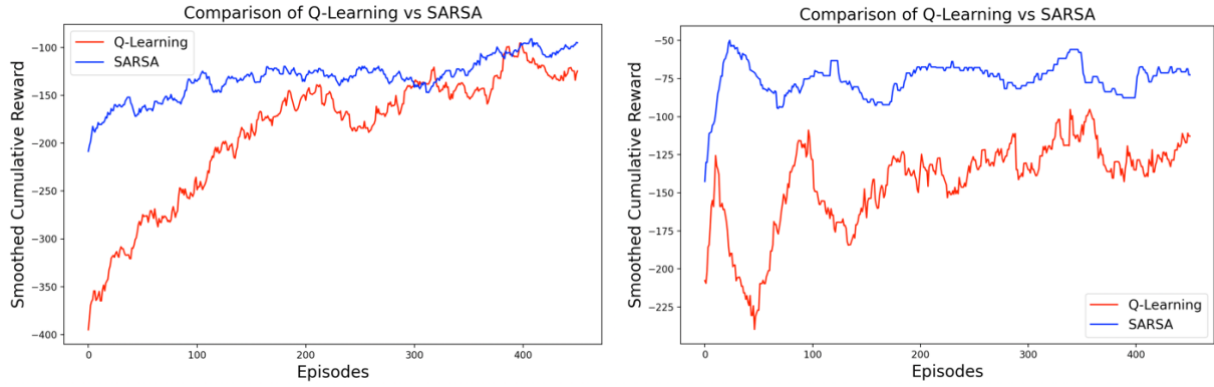


Figure 4.3 Efficiency comparison between Q-Learning and SARSA, $\alpha = 0.5$ (left) and $\alpha = 0.7$ (right)

From the graph, it is evident that Q-Learning is more sensitive to α because it is off-policy. Its updates rely on the maximized Q-values, and when there are large errors, the process tends to be unstable. SARSA, on the other hand, is on-policy, and its updates are based on the actual actions taken. As a result, it is generally more robust to changes in α , leading to a smoother learning process.

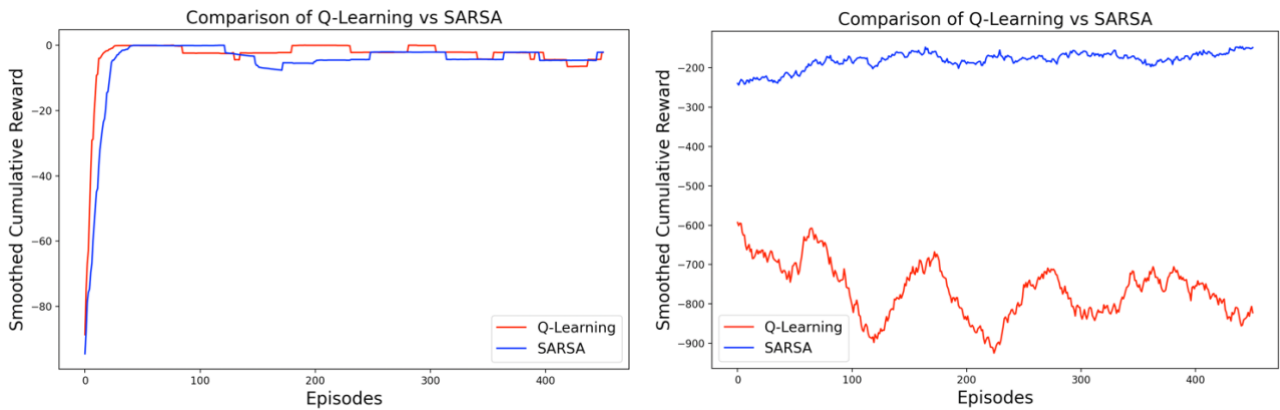


Figure 4.4 Comparison between Q-Learning and SARSA, $\epsilon = 0.01$ (left) and $\epsilon = 0.7$ (right)

Additionally, when setting different values of epsilon, Q-Learning is more dependent on ϵ . If ϵ is too small, it is prone to selecting the wrong maximum value, leading to an inaccurate policy. SARSA, due to its on-policy nature and using actual actions for updates, is less sensitive to ϵ . However, if ϵ is too large, it can still slow down the convergence speed.

Question (e)

In the variant of the `small_world` environment, where the 3 barrier cells randomly contain barriers, the state space is expanded to $\bar{S} = S \times Z_2^3$ increasing the state space size by a factor of 8. This makes it impractical to maintain an explicit Q-table due to the exponential growth in states.

Function approximation is useful here because it allows the agent to generalize across similar states, enabling faster learning by sharing information between states with similar characteristics, such as similar barrier patterns. This reduces memory and computational requirements.

Linear function approximation, however, may not suffice in this environment. Linear models assume a weighted sum of features like this:

$$V(s) = \theta_1 p(s) + \theta_2 \cdot b_1 + \theta_3 \cdot b_2 + \theta_4 \cdot b_3$$

Where b_i is 0 or 1, represent the presence of each barrier. $\theta_1, \theta_2, \theta_3, \theta_4$ are the parameters to be learned. This is not effective at capturing the complex, non-linear dependencies of the barriers.

For example, the combined effect of multiple barriers on optimal action choices cannot be modelled well by a linear function. Non-linearities between nearby states with potential barriers would likely lead to abrupt changes in the value function, which a linear function cannot handle effectively.

In this case, polynomial function approximation might be a better choice:

$$V(s) = \sum_{i=1}^n \theta_i \cdot \phi_i(s)$$

Where $\phi_i(s)$ are the polynomial basis functions of the state s .

Additionally, non-linear basis functions can be introduced to better capture the complex relationships in the environment. For example, higher-degree polynomial functions or radial basis functions (RBF) could be used to model non-linearities more effectively. Alternatively, more advanced techniques like neural networks can be employed for function approximation, allowing the model to learn highly complex and non-linear patterns in the state-action space.